

INTRODUCTION TO SQL

1. What is SQL?

SQL stands for **Structured Query Language**.

SQL is the standard language used to communicate with and manage data stored in relational databases.

In simple terms:

SQL is the language we use to talk to a database.

Think of a database as a person who stores a large amount of information. Instead of speaking English or Filipino to that person, we use **SQL commands** to tell the database what information we need or what changes we want to make.

Example

If we want to display all students from a `students` table:

```
SELECT * FROM students;
```

This means:

"Database, show me all the records stored in the students table."

The important idea is that **SQL allows a user or application to communicate with the database.**

2. What is a Relational Database?

A **relational database** is a database in which information is organized into **tables**.

You can think of a table as similar to an Excel spreadsheet.

For example, we may have a `Students` table:

Student_ID	Name	Course	Year
101	Juan	BSIS	1
102	Maria	BSIS	2

Student_ID	Name	Course	Year
103	Pedro	BSCS	1

A table consists of:

Column

A **column** represents a category or type of information.

Examples:

- Student_ID
- Name
- Course
- Year

Row

A **row** represents one complete record.

For example, the row containing:

101 | Juan | BSIS | 1

represents one student's information.

Table

A **table** is a collection of related records.

The important difference between a simple spreadsheet and a relational database is that a relational database can establish **relationships between multiple tables**.

3. Relationships Between Tables

Suppose we have two tables:

Students

Student_ID	Name
101	Juan
102	Maria

Enrollments

Enrollment_ID	Student_ID	Subject
1	101	SQL
2	102	Programming

Notice that both tables contain:

Student_ID

This common field allows the database to determine which enrollment belongs to which student.

For example:

- Student ID 101 → Juan
- Enrollment Student ID 101 → SQL

Therefore:

Juan is enrolled in SQL.

This relationship between tables is one of the fundamental concepts of a **relational database**.

4. Basic SQL Operations

As a BSIS student, you should first master four fundamental SQL commands:

SQL Command	Purpose
SELECT	Retrieve or get data
INSERT	Add data
UPDATE	Change existing data
DELETE	Remove data

The uploaded material emphasizes these four operations as fundamental SQL commands.

A. SELECT — Retrieve Data

SELECT is used when we want to retrieve or display information from a database.

Example:

```
SELECT Name, Course  
FROM students;
```

This tells the database:

"Show me the name and course of the students."

Another example:

```
SELECT *  
FROM students  
WHERE Course = 'BSIS';
```

This means:

"Show me all students whose course is BSIS."

The `*` means that we want to retrieve all available columns.

B. INSERT — Add Data

`INSERT` is used to add a new record to a table.

Example:

```
INSERT INTO students  
VALUES (104, 'Ana', 'BSIS', 1);
```

This adds a new student:

- Student ID: 104
- Name: Ana
- Course: BSIS
- Year: 1

In simple terms:

INSERT means add a new record.

C. UPDATE — Change Existing Data

`UPDATE` is used when we need to modify information that already exists.

For example, suppose Ana's year level was incorrectly recorded as 1, but she is actually in Year 2.

We can write:

```
UPDATE students  
SET Year = 2  
WHERE Student_ID = 104;
```

This means:

"Change the Year value to 2 for the student whose Student_ID is 104."

Important: Understand WHERE

The `WHERE` clause is extremely important because it identifies **which record should be changed**.

Without a proper condition, you could accidentally modify many records.

D. DELETE — Remove Data

`DELETE` is used to remove records.

Example:

```
DELETE FROM students  
WHERE Student_ID = 104;
```

This means:

"Delete the student whose Student_ID is 104."

Be careful!

If you write:

```
DELETE FROM students;
```

you are telling the database to delete the records from the table without specifying a particular student.

Therefore, always understand the condition before executing `DELETE`.

5. SQL and Database Administration

SQL is not simply a language for searching information.

Database administrators and developers use SQL for many activities, including:

- Retrieving information
- Adding records
- Modifying records
- Deleting records
- Creating tables
- Managing database structures
- Controlling access to data

Therefore:

SQL is a powerful tool for interacting with relational databases.

The material specifically presents SQL as a tool used in database administration and not merely as a search mechanism.

6. The Four Commands You Must Remember

For your examinations, remember this:

SELECT → GET

```
SELECT
```

Get or retrieve data.

INSERT → ADD

INSERT

Add new data.

UPDATE → CHANGE

UPDATE

Change existing data.

DELETE → REMOVE

DELETE

Remove data.

A simple memory technique is:

SELECT = Get
INSERT = Add
UPDATE = Change
DELETE = Remove

SQL JOIN

7. What is a SQL JOIN?

Now we move to one of the most important concepts in relational database management: the **SQL JOIN**.

A **JOIN** allows us to retrieve related information from **two or more tables**.

Why do we need this?

Because a well-designed relational database does not normally put every piece of information into one huge table.

Instead, information is divided into separate tables.

This helps keep the database:

- Organized
- Efficient
- Easier to manage
- Less repetitive

The JOIN allows us to bring related information together when we need it.

8. Example: Employee and Department

Consider these two tables.

Employee Table

emp_id	emp_name	emp_address
101	Juan Dela Cruz	Manila
102	Maria Santos	Pasig
103	Pedro Reyes	Las Piñas

The columns mean:

- `emp_id` → unique employee ID
- `emp_name` → employee's name
- `emp_address` → employee's address

Department Table

dep_id	dep_name	dep_head_id
1	IT	101
2	HR	102
3	Finance	103

The columns mean:

- `dep_id` → unique department ID
 - `dep_name` → department name
 - `dep_head_id` → ID of the employee who heads the department
-

9. How Are the Tables Related?

Look carefully at the following:

Employee

emp_id

Department

dep_head_id

The names of the columns are different, but they represent related information.

For example:

IT → dep_head_id = 101

Then we look at the Employee table:

emp_id = 101 → Juan Dela Cruz

Therefore:

Juan Dela Cruz is the head of the IT Department.

This is the relationship that allows us to perform a JOIN.

10. Primary Key and Foreign Key

These two concepts are extremely important.

Primary Key

A **Primary Key** is a column that uniquely identifies each record in a table.

For the Employee table:

```
emp_id
```

can serve as the Primary Key.

For example:

```
101 → Juan  
102 → Maria  
103 → Pedro
```

Each employee should have a unique employee ID.

Easy way to remember:

Primary Key = "Who am I?"

It uniquely identifies a record.

11. Foreign Key

A **Foreign Key** is a column that references a Primary Key in another table.

In our example:

```
Department.dep_head_id
```

references:

```
Employee.emp_id
```

So we can visualize the relationship as:

```
Employee.emp_id  
    ↑  
    |  
    |  
Department.dep_head_id
```

The Foreign Key creates a connection between related records.

Easy way to remember:

Foreign Key = "Who am I connected to?"

12. Why Do We Need JOIN?

Suppose the manager asks:

"Who is the head of each department?"

If we look only at the Department table, we see:

Department	Head ID
IT	101
HR	102
Finance	103

But we do not immediately see the employee names.

We need to connect the Department table with the Employee table.

That is exactly what a **JOIN** does.

13. INNER JOIN Example

We can write:

```
SELECT Department.dep_name,  
       Employee.emp_name  
FROM Department  
INNER JOIN Employee  
ON Department.dep_head_id = Employee.emp_id;
```

Let's understand this one part at a time.

SELECT

```
SELECT Department.dep_name,  
       Employee.emp_name
```

This tells SQL:

"I want to see the department name and employee name."

FROM

```
FROM Department
```

This tells SQL:

"Start with the Department table."

INNER JOIN

```
INNER JOIN Employee
```

This tells SQL:

"Connect the Department table with the Employee table."

ON

```
ON Department.dep_head_id = Employee.emp_id;
```

This is the condition that tells SQL **how the tables are related**.

It means:

"Match the department head ID with the employee ID."

14. Understanding the Equal Sign

The symbol:

=

means **equal to**.

For example:

101 = 101

is TRUE.

But:

101 = 102

is FALSE.

In the JOIN:

Department.dep_head_id = Employee.emp_id

SQL checks whether the values match.

For example:

101 = 101 → Match
102 = 102 → Match
103 = 103 → Match

When the values match, SQL can connect the records.

15. Result of the JOIN

After executing:

```
SELECT Department.dep_name,  
       Employee.emp_name  
FROM Department
```

```
INNER JOIN Employee
ON Department.dep_head_id = Employee.emp_id;
```

we can obtain:

Department	Department Head
IT	Juan Dela Cruz
HR	Maria Santos
Finance	Pedro Reyes

This is much more useful than simply seeing:

```
IT → 101
HR → 102
Finance → 103
```

The JOIN transforms related information from separate tables into a meaningful result.

16. ON vs WHERE

Students often confuse these two clauses.

Remember:

ON

```
ON Department.dep_head_id = Employee.emp_id
```

ON explains how the tables should be connected.

WHERE

```
WHERE dep_name = 'IT'
```

WHERE filters the resulting records.

For example:

```
SELECT dep_name, emp_name
FROM Department
JOIN Employee
ON Department.dep_head_id = Employee.emp_id
WHERE dep_name = 'IT';
```

Here:

- **ON** → establishes the relationship
- **WHERE** → filters the result

Easy way to remember:

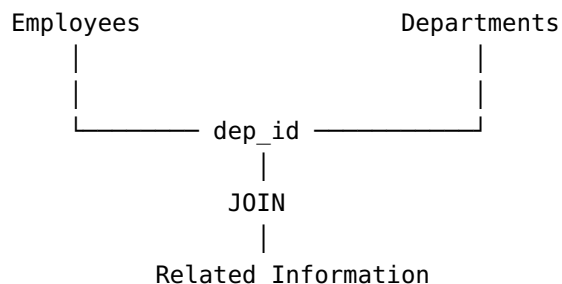
ON = How do the tables connect?

WHERE = Which records do I want?

The material specifically clarifies that the **ON** clause defines the JOIN relationship, while **WHERE** is normally used to filter results.

17. JOIN as a Bridge

As your professor, I want you to visualize a JOIN as a **bridge**.



Think of:

- **Tables** → sources of information
- **Keys** → connection points
- **JOIN** → bridge
- **ON** → rule for connecting the tables

A JOIN does not permanently merge the tables.

Instead, it retrieves related information together when the query is executed.

NATURAL JOIN AND KEY-BASED JOIN

18. What is NATURAL JOIN?

A **NATURAL JOIN** is a type of JOIN that automatically looks for columns with the **same name** and compatible data types in both tables.

For example, suppose both tables contain:

```
emp_id
```

We could write:

```
SELECT dep_name, emp_name  
FROM Department  
NATURAL JOIN Employee;
```

The database automatically uses the common column for matching.

19. Be Careful with NATURAL JOIN

There is an important disadvantage.

Suppose two tables have several columns with the same names:

```
Employee  
emp_id  
name  
address
```

and:

```
Department  
dep_id  
name  
address
```

A NATURAL JOIN may use multiple common columns when determining the match.

Therefore, although NATURAL JOIN can be convenient, an **explicit JOIN condition** is often clearer and gives the developer more control.

This is an important clarification from the lesson.

When a learning material uses the term "KEY JOIN," it may be describing a JOIN that is based on related keys, particularly a:

For example:



This is a **key-based JOIN**, but `KEY JOIN` itself is not normally the SQL syntax used for the operation.

21. NATURAL JOIN vs Explicit JOIN

NATURAL JOIN

```
SELECT dep_name, emp_name
FROM Department
NATURAL JOIN Employee;
```

SQL automatically looks for common column names.

Advantage

- Shorter
- Convenient for simple table designs

Disadvantages

- Less control
- Can cause problems when multiple columns have the same name
- The relationship may be less obvious

Explicit JOIN

```
SELECT dep_name, emp_name
FROM Department
JOIN Employee
ON Department.dep_head_id = Employee.emp_id;
```

Here, we explicitly specify which columns should match.

Advantages

- Clear
- Easier to understand
- More control
- Better for complex databases

For BSIS students, the material recommends learning the **explicit JOIN approach** first.

STANDARD SQL JOIN TYPES

22. INNER JOIN

An **INNER JOIN** returns only records that have matching values in both tables.

Example:

```
SELECT *  
FROM Department  
INNER JOIN Employee  
ON Department.dep_head_id = Employee.emp_id;
```

Think:

"Give me only the records that match."

For example:

```
Students:  
1 → Ana  
2 → Ben  
3 → Carlo  
  
Enrollments:  
1 → Database  
2 → Networking  
4 → Programming
```

The matching IDs are:

```
1  
2
```

Therefore, the INNER JOIN returns information for:

- Ana
- Ben

Carlo is excluded because there is no matching enrollment for Student ID 3.

23. LEFT JOIN

A **LEFT JOIN** returns:

All records from the left table + matching records from the right table.

Example:

```
SELECT *  
FROM Students  
LEFT JOIN Enrollments  
ON Students.student_id = Enrollments.student_id;
```

If Carlo has no enrollment, Carlo is still included.

The enrollment columns for Carlo will contain:

NULL

Remember:

LEFT JOIN → The left table has priority.

24. RIGHT JOIN

A **RIGHT JOIN** is the opposite of a LEFT JOIN.

It returns:

All records from the right table + matching records from the left table.

For example:

```
SELECT *  
FROM Students  
RIGHT JOIN Enrollments  
ON Students.student_id = Enrollments.student_id;
```

All records from the Enrollments table are included, even if there is no matching student.

Remember:

RIGHT JOIN → The right table has priority.

25. FULL JOIN

A **FULL JOIN** returns:

All matching and non-matching records from both tables.

That means:

- Records with matches → included
- Records only in the left table → included
- Records only in the right table → included

When no matching record exists, the columns from the other table may contain `NULL`.

Think:

"Show me everything from both sides."

The material also notes that support for FULL JOIN depends on the database system being used.

26. CROSS JOIN

A **CROSS JOIN** is different from the other JOINS.

It produces every possible combination between the rows of two tables.

For example:

```
3 Departments × 3 Employees  
= 9 combinations
```

SQL:

```
SELECT *  
FROM Department  
CROSS JOIN Employee;
```

If one table has 3 rows and the other has 4 rows:

$3 \times 4 = 12$ rows

So remember:

CROSS JOIN = Everyone with everyone.

A CROSS JOIN can create a very large result, so use it carefully.

27. Quick JOIN Comparison

JOIN Type	What It Returns
INNER JOIN	Only matching records
LEFT JOIN	All left records + matching right records
RIGHT JOIN	All right records + matching left records
FULL JOIN	All records from both tables
CROSS JOIN	Every possible combination

A simple memory guide:

INNER → MATCHING ONLY
LEFT → EVERYTHING ON LEFT
RIGHT → EVERYTHING ON RIGHT
FULL → EVERYTHING ON BOTH SIDES
CROSS → EVERYTHING WITH EVERYTHING

GENERAL SQL JOIN SYNTAX

28. General JOIN Structure

The general structure presented in the material is:

```
FROM table_expression  
[JOIN_OPERATOR] table_expression  
[ON join_condition]
```

Let's understand each part.

A. FROM table_expression

`FROM` tells SQL where the data starts.

Example:

```
FROM employees
```

Meaning:

"Use the employees table as the starting point."

B. JOIN Operator

The JOIN operator tells SQL how the tables should be combined.

Examples:

```
INNER JOIN  
LEFT JOIN  
RIGHT JOIN  
FULL JOIN  
CROSS JOIN
```

Example:

```
FROM employees
INNER JOIN departments
```

Meaning:

"Connect the employees table with the departments table using an INNER JOIN."

C. ON join_condition

The `ON` clause specifies the relationship between the tables.

Example:

```
ON employees.dep_id = departments.dep_id
```

This means:

"Match an employee with a department when their department IDs are equal."

The `dep_id` serves as the connection or bridge between the two tables.

29. Complete Example

Suppose we have:

Employees

emp_id	emp_name	dep_id
101	Juan	10
102	Maria	20
103	Pedro	10

Departments

dep_id	dep_name
10	IT
20	HR

We want to display the employee name and department name.

We write:

```
SELECT employees.emp_name,  
       departments.dep_name  
FROM employees  
INNER JOIN departments  
ON employees.dep_id = departments.dep_id;
```

Let's break it down.

Step 1 — FROM

```
FROM employees
```

Start with the Employees table.

Step 2 — INNER JOIN

```
INNER JOIN departments
```

Connect Employees with Departments.

Step 3 — ON

```
ON employees.dep_id = departments.dep_id
```

Tell SQL how the two tables are related.

Step 4 — Matching

SQL looks for matching `dep_id` values.

The result is:

Employee	Department
Juan	IT
Maria	HR
Pedro	IT

Instead of viewing employee and department information separately, the JOIN creates one useful result containing the related information.

PROFESSOR'S FINAL REVIEW

As a BSIS student, these are the concepts I want you to master.

SQL Fundamentals

SQL
↓
Language for communicating with databases

Four fundamental commands:

SELECT → Get data
INSERT → Add data
UPDATE → Change data
DELETE → Remove data

Database Relationships

PRIMARY KEY
↓
Unique identifier of a record

FOREIGN KEY
↓
References a key in another table

JOIN
↓
Connects related information

JOIN Concepts

SELECT → What do I want to display?

FROM → Where does the data come from?

JOIN → What table should I connect?

ON → How are the tables related?

WHERE → Which records should I filter?

The Most Important Formula

Think of this:

```
TABLE A
  +
TABLE B
  +
RELATED COLUMNS
  ↓
JOIN
  ↓
USEFUL INFORMATION
```

For example:

```
Department.dep_head_id
      =
Employee.emp_id
```

Then:

```
Department + Employee
      ↓
      JOIN
      ↓
IT → Juan Dela Cruz
```

HR → Maria Santos
Finance → Pedro Reyes

★ Final Professor's Takeaway

Do not memorize SQL syntax without understanding **why** you are using it.

When you see:

```
SELECT dep_name, emp_name
FROM Department
JOIN Employee
ON Department.dep_head_id = Employee.emp_id;
```

Think of it as a conversation with the database:

SELECT: "What information do I want?"
FROM: "Where should I start?"
JOIN: "What other table do I need?"
ON: "How are these tables connected?"
WHERE: "Which records do I want to filter?"

And remember the most important concepts:

Primary Key = uniquely identifies a record.
Foreign Key = creates a relationship to another table.
JOIN = brings related information together.
ON = defines how tables are connected.
WHERE = filters the results.

If you understand these concepts, you already have a strong foundation for **Database Management Systems and BSIS Information Systems**.